Arm® FIRME A-profile Architecture Compliance Suite

Version 0.5

Validation Methodology

Non-Confidential

Copyright © 2026 Arm Limited (or its affiliates).
All rights reserved.

Issue 01
111888_0005_01_en

Arm® FIRME A-profile Architecture Compliance Suite Validation Methodology

This document is Non-Confidential.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.

This document is protected by copyright and other intellectual property rights.
Arm only permits use of this document if you have reviewed and accepted Arm's Proprietary Notice found at the end of this document.

This document (111888_0005_01_en) was issued on 2026-03-18. There might be a later issue at https://developer.arm.com/documentation/111888

The product version is 0.5.

See also: Proprietary Notice | Product and document information | Useful resources

Start reading

If you prefer, you can skip to the start of the content.

Intended audience

This book is written for engineers who are specifying, designing, or verifying an implementation of the Arm® FIRME A-profile architecture.

Inclusive language commitment

Arm values inclusive communities. Arm recognizes that we and our industry have used language that can be offensive. Arm strives to lead the industry and create change.

We believe that this document contains no offensive language. To report offensive language in this document, email terms@arm.com.

Feedback

Arm welcomes feedback on this product and its documentation. To provide feedback on the product, create a ticket on https://support.developer.arm.com.

To provide feedback on the document, fill the following survey: https://developer.arm.com/documentation-feedback-survey.

Contents

1. Overview to FIRME ACS.....4
1.1 Abbreviations.....4
1.2 FIRME for Arm A-profile.....4
1.3 FIRME Architecture Compliance Suite.....5
1.4 Source code directory.....5
1.5 Feedback, contributions, and support.....6

2. Validation methodology.....8
2.1 ACS software stack.....8
2.2 Design Under Test.....8
2.3 Test execution flow.....10
2.4 ACS boot flow.....11
2.5 Page table setup.....12
2.6 ACS memory layout.....13
2.7 Test report and analyzing results.....14
2.8 Recovery from faults (expected GPF).....14
2.9 MP execution setup.....15

Proprietary Notice.....16

Product and document information.....18
Product status.....18
Revision history.....18
Conventions.....18

Useful resources.....21

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

1. Overview to FIRME ACS

This chapter describes the validation methodology used by the Firmware Interfaces for RME Architecture Compliance Suite (FIRME ACS) to validate the EL3 implementation of the FIRME service against the requirements defined in the FIRME specification for Arm® FIRME for A-profile.

1.1 Abbreviations

This section lists the abbreviations that are used in this document.

Table 1-1: Abbreviations and expansions

Abbreviation	Expansion
ABI	Application Binary Interface
ACS	Architecture Compliance Suite
DUT	Design Under Test
DSM	Device-Specific Method
EL	Exception Level
GPT	Granule Protection Table
GPI	Granule Protection Indicator
MMU	Memory Management Unit
NS	Non-secure
PAL	Platform Abstraction Layer
RME	Realm Management Extension
RMM	Realm Management Monitor
RCiEP	Root Complex Integrated Endpoint
SPM	Secure Partition Manager
SPMC	Secure Partition Manager Core
SPMD	Secure Partition Manager Dispatcher
VAL	Validation Abstraction Layer
TDISP	Trusted Device Interface Security Protocol
TF-A	Trusted Firmware-A

1.2 FIRME for Arm A-profile

FIRME defines the EL3 firmware interfaces required to support Realm-related operations in systems implementing the Realm Management Extension (RME).

The methodology explains how the test suite exercises FIRME interfaces from different security states and evaluates the behavior of the EL3 firmware implementation.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

While RME provides the architectural mechanisms for Realm isolation, certain system operations require coordination through privileged firmware. FIRME defines standardized ABIs that allow EL3 firmware to:

- Manage Granule Protection Table (GPT) state transitions.
- Refresh Memory Encryption Contexts (MEC) associated with MECIDs.
- Configure and control IDE streams for secure device assignment.
- Support Realm and platform attestation flows.
- Facilitates secure TDISP message transport between RMM and RCiEP DSM.

An Arm implementation of the FIRME specification is also permitted on systems that do not implement FEAT_RME. For example, attestation token management interfaces may be used by the Host to obtain an attestation report from a subsystem that serves as the hardware root of trust.

FIRME provides the privileged firmware control path needed to orchestrate Realm-related platform services and enforce system-level isolation, encryption, and attestation guarantees in an RME-enabled system.

1.3 FIRME Architecture Compliance Suite

FIRME ACS is a self-checking validation suite that verifies the EL3 FIRME implementation by invoking FIRME ABIs from different caller security states and validating the resulting system behavior.

FIRME ACS validates the EL3 implementation by exercising FIRME ABIs from caller security states (Non-secure, Secure, and Realm) and checking observable behavior including return codes, long-running semantics, and enforcement through expected faults.

For more information about ACS as non-exhaustive and non-comprehensive, see the FIRME Architecture Compliance Suite Scenario Document.

1.4 Source code directory

This section describes the directories in the test suite when the release package is downloaded from GitHub.

cca/firme/docs/

Documentation.

cca/firme/platform/:

pal_baremetal/targets/<target>/{include,src}/

PAL target ports. The default target is FVP.

cca/firme/val_common/

hared bare-metal infrastructure used by all worlds, including logging, fault handling, SMCCC wrapper, MMU and GIC helpers, and system register access.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

cca/firme/val/:

host/
: NS-EL2 dispatcher payload (BL33).

secure/
: S-EL2 secure shim payload (BL32).

realm/
: R-EL2 realm shim payload (RMM image slot).

src/ and include/
: World-agnostic VAL wrappers and dispatch.

cca/firme/test/:

test/<module>/
: Rule tests grouped by chapter or module (currently `test/gpt/`).

cca/firme/tools/:

cmake/toolchains/
: Toolchain files.

linker/
: Linker scripts for BL33, BL32, and R-EL2 payload images.

scripts/
: Build and regression scripts.

config/
: SP layout inputs. This directory can be empty if no SPs are packaged.

Note

- Each test under test/src/ corresponds to a single DEN0149 rule.
- World shims reside under `val/{secure,realm}` and are invoked by the Non-secure (NS) host dispatcher.
- Platform overrides are located in `pal_baremetal/targets/<target>/include/pal_platform.h`. This includes configuration for UART, GIC, GPT window, and endpoint identifiers.
- Image base addresses and sizes are defined in `val/*/link.ld` for the FVP and must match the EL3 implementation load addresses.

1.5 Feedback, contributions, and support

For feedback, use the GitHub Issue Tracker that is associated with this repository.

For support, send an email to support-firme-acs@arm.com with the details.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Arm licensees can contact Arm directly through their partner managers.

Arm also welcomes code contributions through GitHub pull requests. See, GitHub documentation on how to raise pull requests.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

2. Validation methodology

This chapter describes the validation methodology for the Architecture Compliance Suite.

2.1 ACS software stack

This section provides information about the layered software stack used by the ACS. The ACS uses the layered software stack approach to enable porting across different test platforms. The layered stack contains:

Tests

Rule-oriented C tests that map 1:1 to DEN0149 rules and use VAL interfaces.
A collection of targeted tests that validate the target system compliance. These tests use interfaces that are provided by the VAL.

VAL

Common runtime infrastructure shared by all the worlds including dispatcher, FIRME ABI wrappers, fault recovery, and cross-world calls.

PAL

The Platform Abstraction Layer (PAL) is an Arm-defined API that you can implement. The API abstracts features whose implementation varies across target systems. Each test platform requires its own PAL implementation. This PAL contains platform configuration and minimal drivers, including UART, memory windows, and endpoint identifiers.

2.2 Design Under Test

The suite validates the EL3 FIRME implementation externally by issuing FIRME SMC calls from each caller security state and checking observable behavior including return codes and fault enforcement.

The following figure shows the caller worlds used by the suite and the EL3 design under test.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Figure 2-1: Design under test

The following table describes the components in the caller worlds used by the suite and the EL3 design under test.

Table 2-1: Components and descriptions

Component	Description
Hardware and platform	Target system (FVP or silicon) that provides CPUs, memory, GPT enforcement, GIC, and the peripheral map used by the suite.
DUT: EL3 firmware	Implementation under test. Provides FIRME SMC handling and EL3 routing to reach Secure and Realm call paths.
NS payload (BL33) - ACS Host (NS-EL2 dispatcher)	Regression anchor. Boots first, runs FIRME probes (VERSION and FEATURES), selects scenarios, runs rule tests, requests Secure and Realm work, validates results, and prints PASS or FAIL for each rule.
Secure payload (BL32) - ACS Secure shim (S-EL2 with optional S-EL1 body)	Secure-side caller context used by tests. Receives host requests through the Secure messaging path through EL3, issues FIRME SMCs from the Secure state, and returns status and fault information.
Realm payload (R-EL2 image) - ACS Realm shim	Realm-side caller context used by tests. Receives host requests through RMMD forwarding through EL3, issues FIRME SMCs from the Realm state, and returns status and fault information.
PAL (Platform Abstraction Layer)	Partner-ported platform inputs and drivers, such as UART and GIC base addresses, GPT test windows, and endpoint IDs. Keeps the suite portable across platforms.
VAL / val_common (Validation Abstraction Layer)	Common run-time infrastructure for all payloads, including logging, SMCCC wrappers, basic bring-up, fault capture and recovery, cross-world call helpers, and reusable test primitives.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Component	Description
Tests	Rule-oriented C tests with a 1:1 mapping to FIRME specification rules. These tests exercise FIRME from expected caller security states and validate observable behavior, including return codes and expected fault or no-fault enforcement.

2.3 Test execution flow

The Non-secure host (NS-EL2) acts as the anchor for regression execution. The host selects test scenarios, requests Secure world and Realm world operations, validates the results, and prints a single PASS or FAIL outcome for each rule.

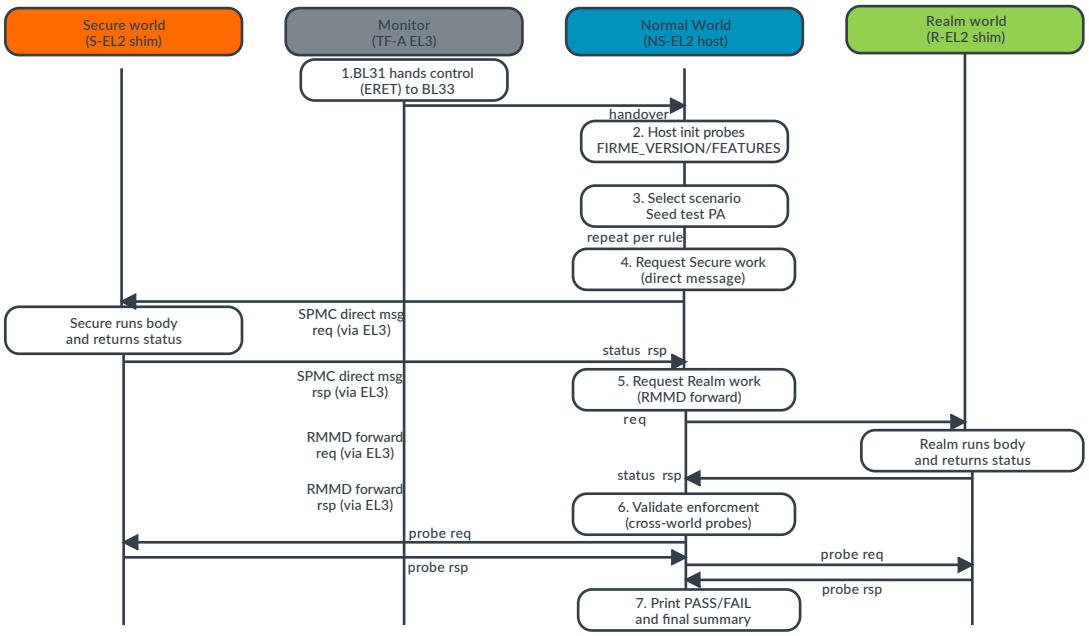
Secure world requests are transported through direct messaging using the SPMC/SPMD path. Realm world requests are transported through the RMMD path using a private RMI/RSI-style dispatch contract.

The numbered steps shown in the test execution flow figure correspond to the regression loop that the host executes:

1. BL31 hands control to the NS payload (BL33), which becomes the regression anchor.
2. The host performs initial FIRME capability discovery by issuing VERSION and FEATURES probes to determine the supported behavior.
3. For each rule, the host selects the appropriate test scenario and seeds the test physical address (PA) within the GPT test window.
4. The host requests Secure world work. The request and response are transported through the SPMC/SPMD path through EL3.
5. The host requests Realm world work. The request and response are transported through the RMMD path through EL3.
6. The host validates enforcement by issuing cross-world probe requests, verifying the expected fault or no-fault behavior.
7. The host prints the PASS or FAIL result for the rule and proceeds to the next rule until all rules are executed.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Figure 2-2: Test execution flow



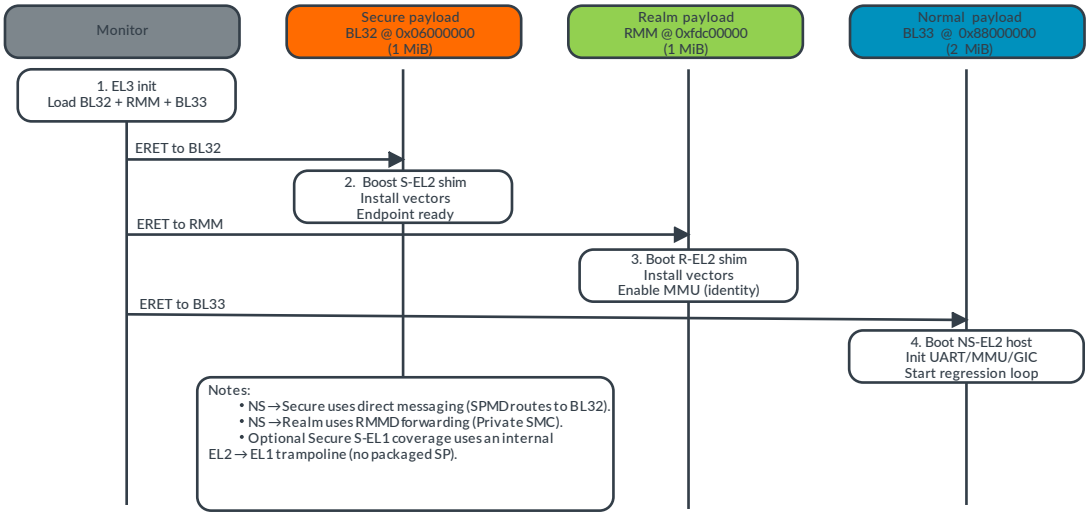
2.4 ACS boot flow

The ACS has common boot flow sequences for Secure, Host, and Realm with each of them having different entry points.

The following figure shows the ACS boot flow.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Figure 2-3: ACS boot flow



The following steps explain the ACS boot flow shown in the figure. The flow describes how control transfers from EL3 firmware to the three FIRME-ACS payloads and how regression begins:

1. EL3 firmware initializes the platform and prepares to launch the non-EL3 payload images.
2. EL3 firmware loads the three FIRME-ACS payload images into their configured physical addresses: BL32 Secure shim, R-EL2 Realm shim image, and BL33 NS-EL2 host.
3. EL3 firmware transfers control (ERET) to the Secure payload (BL32). The Secure shim installs vectors or stack and becomes ready for host requests routed through EL3.
4. EL3 firmware transfers control (ERET) to the Realm payload (R-EL2 image). The Realm shim installs vectors or stack and enables the identity MMU where configured. It then becomes ready for host requests routed through EL3.
5. EL3 firmware transfers control (ERET) to the Normal payload (BL33). The payload enters at NS-EL2 and becomes the regression anchor.
6. The NS-EL2 host initializes UART, MMU, and GIC as required. It runs FIRME probes and starts the rule regression loop. The host requests Secure and Realm work through EL3 routing, validates results, and prints PASS or FAIL.

2.5 Page table setup

The ACS creates a page table for image regions, device regions (for example, UART), and memory pool at runtime as part of the boot sequence.

The harness uses a minimal identity-mapped MMU configuration with a 4KB translation granule and 1GiB block mappings. MMU enablement is currently world-specific.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

NS-EL2 host

Enables the MMU in`nsel2_platform_init()` located in `val/host/main.c`.

R-EL2 shim

Enables the MMU in `rmm_platform_init()` located in `val/realm/main.c`.

S-EL2 shim

Keeps the MMU disabled by default, because BL32 might be loaded outside the DRAM window that is covered by the simple identity mapping. For more information see, `val/secure/spmc.c`.

If the suite is ported to a platform where BL32 resides within DRAM, or if a finer-grained mapping is introduced, the Secure world MMU can be enabled using the same helper functions:

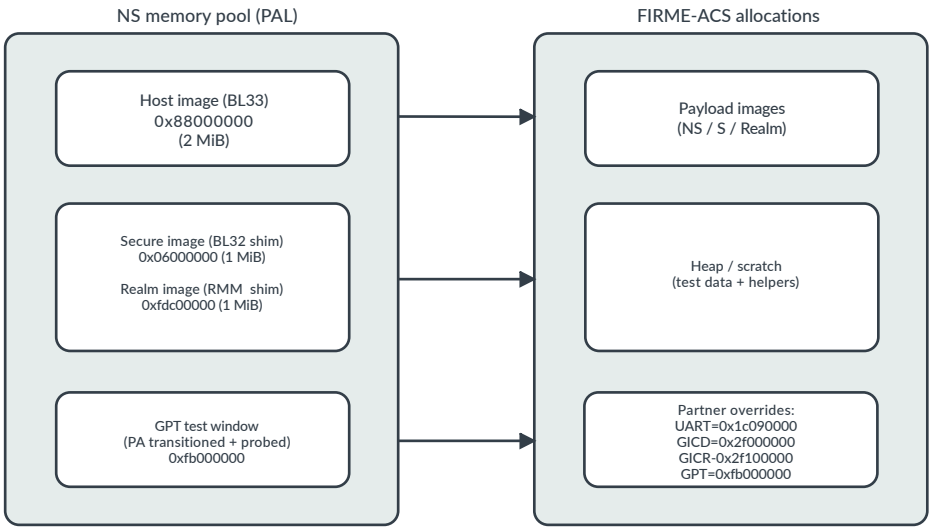
```
mmu_setup_identity_1g() and mmu_enable()
```

2.6 ACS memory layout

The suite uses a deterministic memory layout to simplify debugging and to ensure a stable GPT test window for Granule Protection Indicator (GPI) transitions and probe operations.

The following figure shows Memory layout (conceptual).

Figure 2-4: Memory layout (conceptual)



Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Platform configuration parameters

The following table lists platform-dependent addresses that users typically override. These definitions are provided in `pal/include/pal_platform.h`.

Table 2-2: Platform configuration parameters

Name	Default	Purpose
PLAT_NS_UART_BASE	0x1c090000	UART base address used for logs (Non-secure console).
PLAT_GICD_BASE	0x2f000000	Generic Interrupt Controller (GIC) Distributor base address.
PLAT_GICR_BASE	0x2f100000	Generic Interrupt Controller (GIC) Re-distributor base address.
PLAT_FIRME_GPT_BASE_NS	0xfb000000	Base physical address used for GPT transition tests.
PLAT_FIRME_GPT_GRANULE_SIZE	0x00001000	GPT granule size (typically 4KB).

2.7 Test report and analyzing results

The standard output is printed by the Non-secure (NS) dispatcher. The output includes a suite banner, per-test header, checkpoint logs, and a final summary of the test results.

An example test suite summary is as follows:

```
*****************************************************
Suite: firme, Test: gpt_transition_policy
Rule: R0086
Desc: FEAT_RME_GPC2 additional NS<->NSO transitions

Check 1: GM_GPI_SET from NS (NS<->NSO)
Result: PASS
*****************************************************

-------------------------------------------------------------
Total Tests run = 3 ; Tests Passed = 3 ; Tests Failed = 0
-------------------------------------------------------------
```

2.8 Recovery from faults (expected GPF)

Rule tests often validate enforcement by performing accesses that are expected to generate faults in certain worlds.

To prevent the suite from hanging when such faults occur, each payload installs exception vectors and records the associated fault syndrome information, including ESR and FAR values, before returning control to the dispatcher.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

2.9 MP execution setup

The current harness focuses on Uniprocessor (UP) execution on the FVP.

When Multiprocessor (MP) execution is enabled, the implementation must preserve the same world-switching contracts and ensure that per-core shared state remains explicit, for example by maintaining shared region offsets per CPU.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Proprietary Notice

This document is protected by copyright and other related rights and the use or implementation of the information contained in this document may be protected by one or more patents or pending patent applications. No part of this document may be reproduced in any form by any means without the express prior written permission of Arm Limited ("Arm"). No license, express or implied, by estoppel or otherwise to any intellectual property rights is granted by this document unless specifically stated.

Your access to the information in this document is conditional upon your acceptance that you will not use or permit others to use the information for the purposes of determining whether the subject matter of this document infringes any third party patents.

The content of this document is informational only. Any solutions presented herein are subject to changing conditions, information, scope, and data. This document was produced using reasonable efforts based on information available as of the date of issue of this document. The scope of information in this document may exceed that which Arm is required to provide, and such additional information is merely intended to further assist the recipient and does not represent Arm's view of the scope of its obligations. You acknowledge and agree that you possess the necessary expertise in system security and functional safety and that you shall be solely responsible for compliance with all legal, regulatory, safety and security related requirements concerning your products, notwithstanding any information or support that may be provided by Arm herein. In addition, you are responsible for any applications which are used in conjunction with any Arm technology described in this document, and to minimize risks, adequate design and operating safeguards should be provided for by you.

This document may include technical inaccuracies or typographical errors. THIS DOCUMENT IS PROVIDED "AS IS". ARM PROVIDES NO REPRESENTATIONS AND NO WARRANTIES, EXPRESS, IMPLIED OR STATUTORY, INCLUDING, WITHOUT LIMITATION, THE IMPLIED WARRANTIES OF MERCHANTABILITY, SATISFACTORY QUALITY, NON-INFRINGEMENT OR FITNESS FOR A PARTICULAR PURPOSE WITH RESPECT TO THE DOCUMENT. For the avoidance of doubt, Arm makes no representation with respect to, and has undertaken no analysis to identify or understand the scope and content of, any patents, copyrights, trade secrets, trademarks, or other rights.

TO THE EXTENT NOT PROHIBITED BY LAW, IN NO EVENT WILL ARM BE LIABLE FOR ANY DAMAGES, INCLUDING WITHOUT LIMITATION ANY DIRECT, INDIRECT, SPECIAL, INCIDENTAL, PUNITIVE, OR CONSEQUENTIAL DAMAGES, HOWEVER CAUSED AND REGARDLESS OF THE THEORY OF LIABILITY, ARISING OUT OF ANY USE OF THIS DOCUMENT, EVEN IF ARM HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

Reference by Arm to any third party's products or services within this document is not an express or implied approval or endorsement of the use thereof.

This document consists solely of commercial items. You shall be responsible for ensuring that any permitted use, duplication, or disclosure of this document complies fully with any relevant export laws and regulations to assure that this document or any portion thereof is not exported,

directly or indirectly, in violation of such export laws. Use of the word “partner” in reference to Arm’s customers is not intended to create or refer to any partnership relationship with any other company. Arm may make changes to this document at any time and without notice.

This document may be translated into other languages for convenience, and you agree that if there is any conflict between the English version of this document and any translation, the terms of the English version of this document shall prevail.

The validity, construction and performance of this notice shall be governed by English Law.

The Arm corporate logo and words marked with ® or ™ are registered trademarks or trademarks of Arm Limited (or its affiliates) in the US and/or elsewhere. Please follow Arm’s trademark usage guidelines at https://www.arm.com/company/policies/trademarks. All rights reserved. Other brands and names mentioned in this document may be the trademarks of their respective owners.

Arm Limited. Company 02557590 registered in England.

110 Fulbourn Road, Cambridge, England CB1 9NJ.

PRE-1121-V1.0

Product and document information

Read the information in these sections to understand the release status of the product and documentation, and the conventions used in the Arm documents.

Product status

All products and Services provided by Arm require deliverables to be prepared and made available at different levels of completeness. The information in this document indicates the appropriate level of completeness for the associated deliverables.

Product completeness status

The information in this document is for an Alpha product, that is a product under development.

Revision history

These sections can help you understand how the document has changed over time.

Document release information

The Document history table gives the issue number and the released date for each released issue of this document.

Document history

Issue	Date	Confidentiality	Change
0005-01	18 March 2026	Non-Confidential	v0.5 Alpha release

The Change history tables describe the technical changes between released issues of this document in reverse order. Issue numbers match the revision history in Document release information on page 18.

Table 2: Issue 0005-01

Change	Location
First release.	-

Conventions

The following subsections describe conventions used in Arm documents.

Glossary

The Arm Glossary is a list of terms used in Arm documentation, together with definitions for those terms. The Arm Glossary does not contain terms that are industry standard unless the Arm meaning differs from the generally accepted meaning.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

See the Arm Glossary for more information: developer.arm.com/glossary.

Typographic conventions

Arm documentation uses typographical conventions to convey specific meaning.

Convention	Use
italic	Citations.
bold	Interface elements, such as menu names. Terms in descriptive lists, where appropriate.
`monospace`	Text that you can enter at the keyboard, such as commands, file and program names, and source code.
`monospace` <u>`underline`</u>	A permitted abbreviation for a command or option. You can enter the underlined text instead of the full command or option name.
`<and>`	Encloses replaceable terms for assembler syntax where they appear in code or code fragments. For example: `MRC p15, 0, <Rd>, <CRn>, <CRm>, <Opcode_2>`
SMALL CAPITALS	Terms that have specific technical meanings as defined in the *Arm® Glossary*. For example, IMPLEMENTATION DEFINED, IMPLEMENTATION SPECIFIC, UNKNOWN, and UNPREDICTABLE.

Caution

We recommend the following. If you do not follow these recommendations your system might not work.

Warning

Your system requires the following. If you do not follow these requirements your system will not work.

Danger

You are at risk of causing permanent damage to your system or your equipment, or of harming yourself.

Note

This information is important and needs your attention.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Tip

This information might help you perform a task in an easier, better, or faster way.

Remember

This information reminds you of something important relating to the current content.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Useful resources

This document contains information that is specific to this product. See the following resources for other useful information.

Arm documents are available on developer.arm.com/documentation.

Confidential documents are only available to licensees, when logged in. Each document link in the following tables provides direct access to the online version of the document.

Arm product resources	Document ID	Confidentiality
Firmware Interfaces for Realm Management Extension	DEN0149	Non-Confidential
Arm® Architecture Reference Manual for A-profile architecture	DDI0487	Non-Confidential
Arm® SMC Calling Convention	DEN0028D	non-Confidential
Arm®Realm Management Monitor specification	DEN0137	non-Confidential
Arm® FF-A Memory Management Protocol	DEN0140	Non-Confidential

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential